

Listing 1 航段能耗与最大安全载荷计算

```

293 def equivalent_range_m(g: TransportType, payload_kg: float) -> float:
294     ratio = min(max(payload_kg / g.payload_kg, 0.0), 1.0)
295     return g.empty_range_m - (g.empty_range_m - g.full_range_m) * ratio ** 1.5
296
297
298 def leg_metrics(dem: DemGrid, a: Node, b: Node, g: TransportType, payload_kg: float) -> dict[str, float]:
299     distance = horizontal_distance_m(a.lon, a.lat, b.lon, b.lat)
300     _, elev, _ = dem.path_elevations(a.lon, a.lat, b.lon, b.lat)
301     cruise = float(np.max(elev) + 50.0)
302     climb = max(0.0, cruise - a.operation_m)
303     descent = max(0.0, cruise - b.operation_m)
304     flight_s = climb / g.climb_mps + distance / g.cruise_mps + descent / g.descent_mps
305     horizontal_kwh = g.energy_kwh * distance / equivalent_range_m(g, payload_kg)
306     climb_kwh = (g.empty_mass_kg + payload_kg) * G0 * climb / (3.6e6 * g.climb_eff)
307     return {
308         "distance_m": distance,
309         "cruise_alt_m": cruise,
310         "climb_m": climb,
311         "descent_m": descent,
312         "time_s": flight_s,
313         "energy_kwh": horizontal_kwh + climb_kwh,
314     }
315
316
317 def direct_round_trip(dem: DemGrid, depot: Node, service: Node, g: TransportType, payload_kg: float) -> dict[
    str, float]:
318     outbound = leg_metrics(dem, depot, service, g, payload_kg)
319     inbound = leg_metrics(dem, service, depot, g, 0.0)
320     energy = outbound["energy_kwh"] + inbound["energy_kwh"]
321     return {
322         "energy_kwh": energy,
323         "flight_s": outbound["time_s"] + inbound["time_s"],
324         "return_soc": 1.0 - energy / g.energy_kwh,
325         "outbound_s": outbound["time_s"],
326         "inbound_s": inbound["time_s"],
327         "outbound_cruise_m": outbound["cruise_alt_m"],
328         "inbound_cruise_m": inbound["cruise_alt_m"],
329     }
330
331
332 def max_safe_payload(dem: DemGrid, depot: Node, service: Node, g: TransportType) -> float:
333     limit = (1.0 - g.reserve) * g.energy_kwh
334     if direct_round_trip(dem, depot, service, g, 0.0)["energy_kwh"] > limit:
335         return float("nan")
336     if direct_round_trip(dem, depot, service, g, g.payload_kg)["energy_kwh"] <= limit:
337         return g.payload_kg
338     lo, hi = 0.0, g.payload_kg
339     for _ in range(60):
340         mid = (lo + hi) / 2.0
341         if direct_round_trip(dem, depot, service, g, mid)["energy_kwh"] <= limit:
342             lo = mid
343         else:
344             hi = mid
345     return lo

```

代码清单2给出单服务区可行子集枚举与集合划分动态规划。候选必须同时满足质量、体积和返航能量约束；状态中保存当前最优完整路径，最低有效位锚定用于消除同一划分的排列重复。

Listing 2 问题一可行箱组枚举与动态规划

```

357 def batch_metrics(
358     dem: DemGrid, depot: Node, service: Node, g: TransportType, batch: pd.DataFrame
359 ) -> dict[str, float]:
360     mass = float(batch["单箱质量 (kg)"].sum())
361     volume = float(batch["单箱体积 (m³)"].sum())
362     trip = direct_round_trip(dem, depot, service, g, mass)
363     duration = (
364         g.prep_s
365         + len(batch) * g.load_per_box_s
366         + trip["flight_s"]
367         + g.base_handover_s
368         + len(batch) * g.handover_per_box_s

```